AeroTune Install & Run Guide

Updated for AeroTune V1.6.1

AeroTune is a local-first FPV Blackbox analysis tool for Betaflight tuning support. It reads flight-log data, analyzes roll/pitch/yaw behavior, and returns conservative tuning guidance instead of pretending to calculate perfect final PID values.

1. Recommended Workflow: CSV First

The recommended workflow is:

```
Betaflight raw Blackbox log
        -> open in Betaflight Blackbox Explorer
        -> choose the correct flight
        -> export that flight as CSV
        -> upload CSV into AeroTune
```

This is recommended because raw `.BBL` files can contain multiple flights. Blackbox Explorer lets you select the flight you actually want to analyze before exporting. If the log contains seven flights, the newest one is typically shown as `7/7`. If the log contains three flights, the newest one is typically `3/3`.

Selecting the correct/latest flight before CSV export prevents AeroTune from analyzing the wrong internal flight.

2. Supported Upload Types

AeroTune supports:

```
.csv  - recommended, analyzed directly
.bbl  - raw Blackbox log, converted locally if blackbox_decode is installed
.bfl  - raw Blackbox log, converted locally if blackbox_decode is installed
.txt  - raw Blackbox text log, converted locally if blackbox_decode is installed
```

CSV files work without extra tools. Raw files require `blackbox_decode` to be installed locally or pointed to with `BLACKBOX_DECODE_PATH`.

AeroTune does not bundle or redistribute `blackbox_decode`. It calls your local copy when raw conversion is needed.

3. Local Requirements

Required:

- Python 3
- pip
- Git
- AeroTune repository files

Optional for raw log upload:

- Betaflight `blackbox_decode`
- `make` and compiler tools if building blackbox-tools locally

4. macOS / Linux Setup

```
git clone https://github.com/bostromdev/AeroTune.git
cd AeroTune
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install --upgrade pip
python3 -m pip install -r requirements.txt
python3 -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

Open:

```
http://127.0.0.1:8000
```

5. Windows PowerShell Setup

```
git clone https://github.com/bostromdev/AeroTune.git
cd AeroTune
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
py -3 -m pip install --upgrade pip
py -3 -m pip install -r requirements.txt
py -3 -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

Open:

```
http://127.0.0.1:8000
```

If PowerShell blocks the activation script, run:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Then activate again:

```
.\.venv\Scripts\Activate.ps1
```

6. Windows Command Prompt Setup

```
git clone https://github.com/bostromdev/AeroTune.git
cd AeroTune
py -3 -m venv .venv
.venv\Scripts\activate.bat
py -3 -m pip install --upgrade pip
py -3 -m pip install -r requirements.txt
py -3 -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

7. Windows Troubleshooting: Uvicorn Installed But Not Found

If Windows says `uvicorn` is not recognized, but pip says `Requirement already satisfied`, use:

```
py -3 -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

This runs uvicorn through Python and avoids PATH issues.

8. Optional Raw Blackbox Converter Setup

Use this only if you want AeroTune to accept raw `.BBL`, `.BFL`, or `.TXT` logs directly.

AeroTune searches for `blackbox_decode` in this order:

```
1. BLACKBOX_DECODE_PATH
2. tools/blackbox-tools/obj/blackbox_decode
3. system PATH
```

If raw upload fails, export CSV from Betaflight Blackbox Explorer and upload that CSV instead.

9. Planned Raw Multi-Flight Selector

A future AeroTune feature can decode multiple flights from one raw `.BBL` file and show a selector like:

```
Flight 1/7
Flight 2/7
Flight 3/7
...
Flight 7/7  <- newest/default
```

The tuning advisor would build one profile per decoded flight. The UI should default to the highest-numbered flight because that is normally the most recent flight in the log.

Until that is implemented, use Blackbox Explorer to select the exact flight and export CSV.

10. Recommended Blackbox Test Flight

Use a repeatable 60-90 second test flight:

- Smooth cruise
- Small roll, pitch, and yaw inputs
- Controlled throttle punches
- Medium turns
- Quick stops and direction changes
- One or two safe dirty-air / propwash recovery moments

Avoid logs dominated by crashes, heavy wind, damaged props, battery sag, or random hovering.

11. Single-Log Analysis

Use single-log analysis to answer:

```
What is wrong?
Which axis is affected?
Which PID/filter direction should I try?
Why is that recommendation being made?
```

Steps:

1. Export a Betaflight CSV, or upload a raw log if `blackbox_decode` is installed.
2. Choose drone size.
3. Choose tuning goal.
4. Upload the log.
5. Review PID Tuning Advice.
6. Enter current Betaflight PID values to calculate suggested new values.
7. Make one change pass only.
8. Fly again and compare.

12. Tune-Change Tracking

Use tune-change tracking when you want to validate a specific change:

```
1. Fly a before log.
2. Make one clear PID/filter/rate change.
3. Fly the same test again.
4. Upload before and after logs.
5. Write what changed.
6. Review the keep/reduce/revert/retest result.
```

13. Privacy and Local-Only Notes

AeroTune is designed for local use because FPV Blackbox files can be large and can contain personal flight-analysis details. Local use avoids hosted upload limits, request timeouts, and unnecessary upload of private logs to a public server.

14. License Reminder

AeroTune is source-available for personal non-commercial local evaluation. See `LICENSE` and `NOTICE` before copying, hosting, redistributing, modifying, forking, public release, commercializing, or reusing AeroTune code, logic, UI, docs, branding, or project materials.